

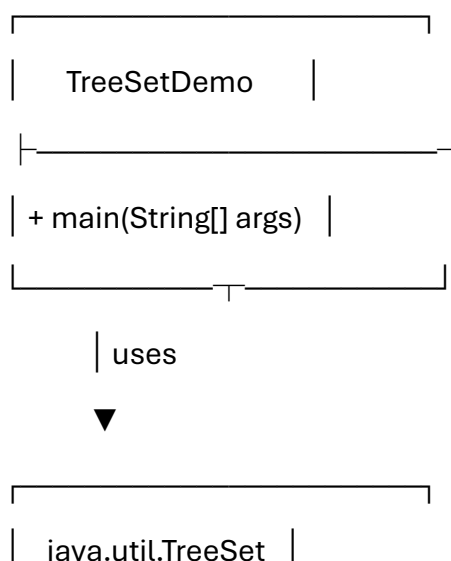
Aim

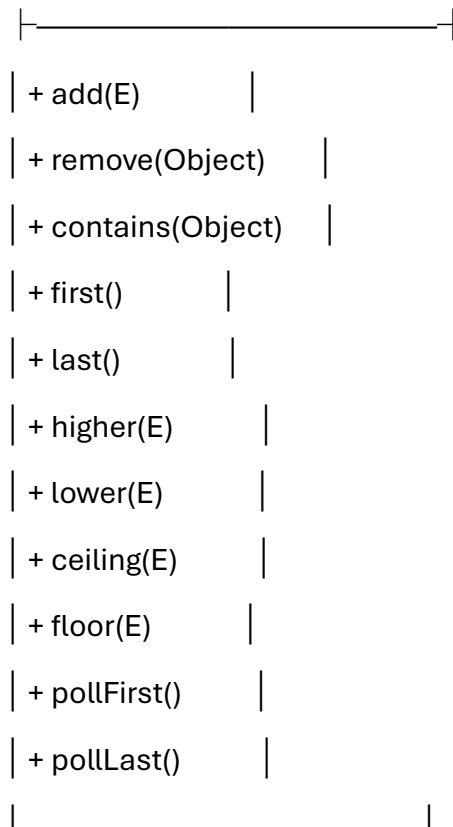
To write a Java program to demonstrate the important methods of the **TreeSet class**, such as `add()`, `remove()`, `contains()`, `first()`, `last()`, `higher()`, `lower()`, `ceiling()`, `floor()`, `pollFirst()`, and `pollLast()`.

Algorithm

1. Start the program.
2. Import the `java.util.TreeSet` class.
3. Create a `TreeSet` object.
4. Add elements using `add()`.
5. Display the elements in sorted order.
6. Check an element using `contains()`.
7. Remove an element using `remove()`.
8. Find the lowest and highest elements using `first()` and `last()`.
9. Find the elements immediately greater and smaller using `higher()` and `lower()`.
10. Find the elements greater than/equal and smaller than/equal using `ceiling()` and `floor()`.
11. Remove and retrieve the first and last elements using `pollFirst()` and `pollLast()`.
12. Display all results.
13. Stop the program.

UML Diagram





Code:

```
package Observations;
```

```
import java.util.TreeSet;
```

```
public class TreeSetClass {
```

```
    public static void main(String[] args) {
```

```
        // Creating TreeSet
```

```
        TreeSet<Integer> set = new TreeSet<>();
```

```
        // add()
```

```
        set.add(10);
```

```
        set.add(20);
```

```
        set.add(30);
```

```
        set.add(40);
```

```
set.add(50);
```

```
System.out.println("TreeSet after add(): " + set);
```

```
// contains()
```

```
System.out.println("Contains 30: " + set.contains(30));
```

```
// remove()
```

```
set.remove(20);
```

```
System.out.println("TreeSet after remove(20): " + set);
```

```
// first() and last()
```

```
System.out.println("First element: " + set.first());
```

```
System.out.println("Last element: " + set.last());
```

```
// higher() and lower()
```

```
System.out.println("Higher than 30: " + set.higher(30));
```

```
System.out.println("Lower than 30: " + set.lower(30));
```

```
// ceiling() and floor()
```

```
System.out.println("Ceiling of 35: " + set.ceiling(35));
```

```
System.out.println("Floor of 35: " + set.floor(35));
```

```
// pollFirst()
```

```
System.out.println("Removed first element: " + set.pollFirst());
```

```
System.out.println("TreeSet after pollFirst(): " + set);
```

```
// pollLast()
```

```
        System.out.println("Removed last element: " + set.pollLast());

        System.out.println("TreeSet after pollLast(): " + set);
    }
}
```

Output:

```
bservations.TreeSetClass'
TreeSet after add(): [10, 20, 30, 40, 50]
Contains 30: true
TreeSet after remove(20): [10, 30, 40, 50]
First element: 10
Last element: 50
Higher than 30: 40
Lower than 30: 10
Ceiling of 35: 40
Floor of 35: 30
Removed first element: 10
TreeSet after pollFirst(): [30, 40, 50]
Removed last element: 50
TreeSet after pollLast(): [30, 40]
PS C:\Users\Sahithi\pyla\Desktop\Sahithi_java>
```

Result:

Thus, the Java program successfully demonstrates the TreeSet methods and shows that TreeSet maintains its elements in sorted order.